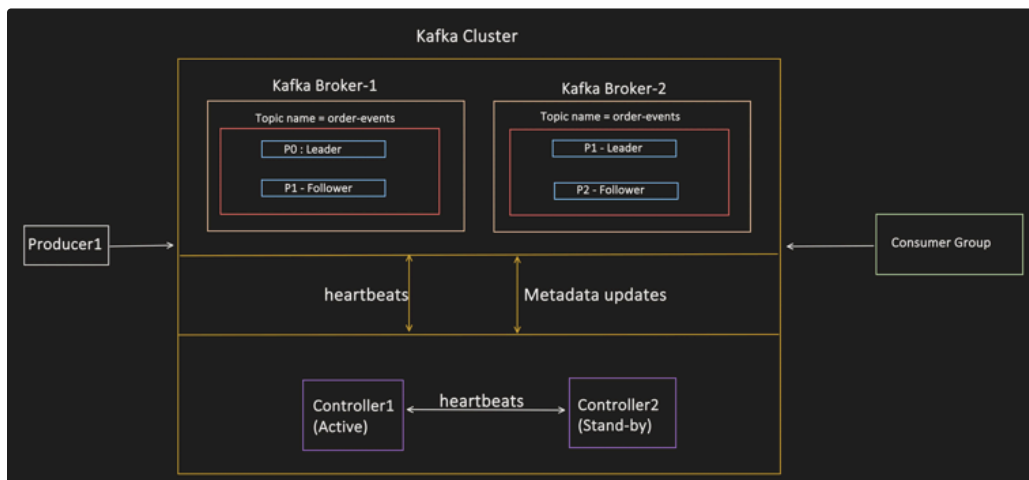


Cluster Setup

Today, will setup below Kafka Cluster.



Step1: Download Binary of latest Kafka Version

Note:

Instead of downloading Binary, we can also go with Kafka Docker Image. But since I haven't covered Docker yet, for now will show how to do with Binary download.

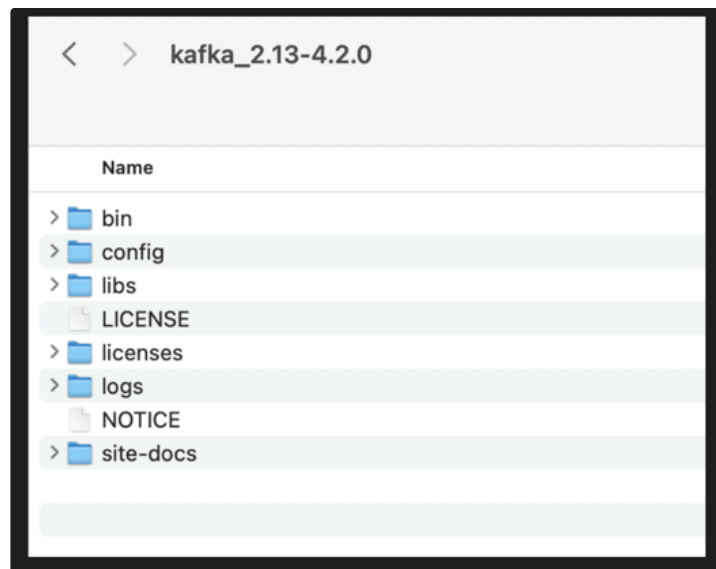
Which is also good way to understand things and once understood properly, Running via Docker image is no different and would be very easy.

Lets download latest Kafka Binary version from : [Downloads](#)

I choose 4.2.0, Which is latest and released on 17th Feb 2026

Binary download: [kafka_2.13-4.2.0.tgz](#) ([asc](#), [sha512](#))

Once downloaded, you will see below sub-folders within Kafka folder.



Step2: Understand each sub-folder



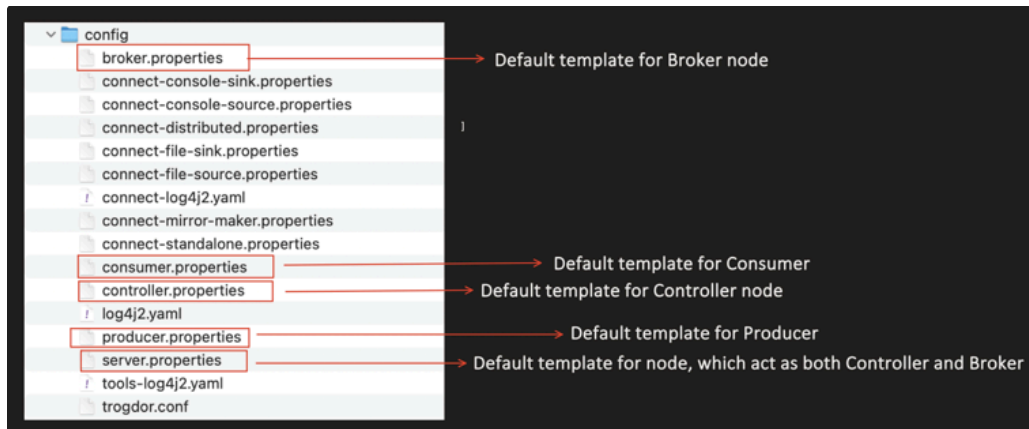
This folder contains executable shell scripts, which is used to perform tasks like:

- Start controller server
- Start broker server
- Create topic
- Delete topic
- Publish message

- Consume message
etc..

These scripts are wrappers around Kafka internal Java APIs. So that we can run and manage kafka components from CLI itself.

Config: This folder is important for us, as it contains all the configuration for our Controller and Broker nodes.



Other folders are like:

libs: contains the kafka jar files and their dependencies. Scripts within bin/ uses these JARs.

logs: Contains Kafka runtime/application logs (INFO, WARN, ERROR messages) generated when brokers or controllers start and run.

Step3: Write the configuration for Controllers

These configuration file need to put in /config folder.

controller1.properties

```
1 process.roles=controller
2
3
4 node.id=1
5
6 listeners=CONTROLLER://:9093
7
8 controller.listener.names=CONTROLLER
9
10 controller.quorum.voters=1@localhost:9093,2@localhost:9193
```

```

11
12 # PLAINTEXT (no encryption). In production, we should use SSL.
13 listener.security.protocol.map=CONTROLLER:PLAINTEXT
14
15 log.dirs=/tmp/controller1-logs
16
17 # Default partitions when creating a topic without specifying
18 num.partitions=3
19 default.replication.factor=2

```

controller2.properties

```

1 process.roles=controller
2
3 node.id=2
4
5 listeners=CONTROLLER://:9193
6
7 controller.listener.names=CONTROLLER
8
9 controller.quorum.voters=1@localhost:9093,2@localhost:9193
10
11 # PLAINTEXT (no encryption). In production, we should use SSL.
12 listener.security.protocol.map=CONTROLLER:PLAINTEXT
13
14 log.dirs=/tmp/controller2-logs
15
16 # Default partitions when creating a topic without specifying
17 num.partitions=3
18 default.replication.factor=2
19

```

process.roles → tells kafka what role this node should perform:

Roles:

- **controller** : only perform controller tasks
- **broker** : only perform broker tasks
- **controller, broker** : performs tasks of both controller and broker.

node.id → unique id for the node within a cluster.

listeners → list of Port which are running on this node, also for each port we can give logical name.

internally it does something like:

new ServerSocket(9093), a TCP socket waiting for connections.

```
1 listeners=CONTROLLER://:9093,BROKER://:9092
```

This tells Kafka, open 2 sockets:

- One named CONTROLLER
- One named BROKER

listeners=CONTROLLER://:9093

“CONTROLLER” : *This is just a logical name (label) for this socket.*

“:9093”: *If IP is omitted, kafka will listen on every IP address on the machine.*

if you want any specific IP address: CONTROLLER://192.168.1.10:9093

controller.listener.names →

```
1 listeners=CONTROLLER://:9093, INTERNAL://:9092, EXTERNAL://:9094
2
3 controller.listener.names=CONTROLLER
4
```

This tells, among all the listeners defined, use the one with named "CONTROLLER" for controller-controller communication.

controller.quorum.voters → each controller, maintains a static list of nodes which act as controller

controller.quorum.voters=1@localhost:9093,2@localhost:9193

listener.security.protocol.map → For which listener, what security protocol need to be used.

PLAINTEXT - no encryption, no authentication, used for local

SSL - TLS encryption, used in production

listener.security.protocol.map=CONTROLLER:PLAINTEXT

(for controller listener, use PLAINTEXT security protocol)

log.dirs → directory where node stores its data.

num.partitions → while during topic creation, if producer or CLI do not tell how many partitions, then controller use this default value.

default.replication.factor → default value for RF, how many replication for each partition.

offsets.topic.replication.factor → default value for _consumer_offset topic

Step4: Write the configuration for Brokers

These configuration file need to put in /config folder.

broker1.properties

```
1 process.roles=broker
2
3 node.id=3
4
5 # WHERE to fetch metadata from (which controllers to connect to)
6 controller.quorum.voters=1@localhost:9093,2@localhost:9193
7
8 # This is where our Spring Boot apps(producer/consumer) will connect
9 listeners=BROKER://:9092
10
11 # passed this address to controller to store in their cluster metadata
12 advertised.listeners=BROKER://localhost:9092
13
14 # internally when follower broker try to fetch from leader, what
15 # listener they need to use.
16 inter.broker.listener.name=BROKER
17
18 # when broker try to communicate with controller what listener they
19 # need to use.
20 controller.listener.names=CONTROLLER
21
22 listener.security.protocol.map=CONTROLLER:PLAINTEXT,BROKER:PLAINTEXT
23
24 log.dirs=/tmp/broker1-logs
25
26 # Keep messages for 7 days (168 hours)
27 log.retention.hours=168
28
29 # Max size of a single log segment file (1GB)
30 log.segment.bytes=1073741824
31
32 # __consumer_offsets topic RF
33 offsets.topic.replication.factor=2
34
35
```

Similarly, broker2.properties

```
1 process.roles=broker
2
3 node.id=4
4
5 # WHERE to fetch metadata from (which controllers to connect to)
6 controller.quorum.voters=1@localhost:9093,2@localhost:9193
7
8 # This is where our Spring Boot apps(producer/consumer) will connect
9 listeners=BROKER://:9192
10
11 # passed this address to controller to store in their cluster metadata
12 advertised.listeners=BROKER://localhost:9192
13
14 # internally when follower broker try to fetch from leader, what
15 # listener they need to use.
16 inter.broker.listener.name=BROKER
17
18 # when broker try to communicate with controller what listener they
19 # need to use.
20 controller.listener.names=CONTROLLER
21
22 listener.security.protocol.map=CONTROLLER:PLAINTEXT,BROKER:PLAINTEXT
23
24 #Directory in which this node data will persist.
25 log.dirs=/tmp/broker2-logs
26
27 # Keep messages for 7 days (168 hours)
28 log.retention.hours=168
29
30 # Max size of a single log segment file (1GB)
31 log.segment.bytes=1073741824
32
33 # __consumer_offsets topic RF
34 offsets.topic.replication.factor=2
```

controller.quorum.voters: Each broker also maintains the static list of controller nodes

listeners: Opens up the port and named it BROKER.

advertised.listeners: This address is passed to controller. And controller stored this address in its metadata.

when producer want to push to Topic-A and Partition0,

it will ask any Broker → Broker goes to Controller, Controller returns the metadata,

which has all the details like which broker is the leader of which topic partition.

and this address, so that producer can directly call that specific Broker.

inter.broker.listener.name: This tells, among all the listeners defined, use the one with named "BROKER" for broker-broker communication.

As we know follower broker talk with leader broker to fetch the data.

controller.listener.names:

2 use:

- Broker use this Controller listener name to map the Security protocol.
- Fetch the controller listener from the metadata

lets say in cluster metadata this info is present:

```
1 {
2     "controllerId": 1,
3     "endPoints": [
4         {
5             "name": "CONTROLLER",
6             "host": "localhost",
7             "port": 9093,
8             "securityProtocol": 0
9         },
10        {
11            "name": "INTERNAL",
12            "host": "localhost",
13            "port": 9097,
14            "securityProtocol": 1
15        }
16    ]
17 }
18 }
```

say broker has decided, it need to communicate with Controller with ID:1 (as its ACTIVE), so it will pick the listener with name "CONTROLLER"

And that's one reason why all different controllers have same listener name.

Step5: map all these nodes ids (2 broker and 2 controller) with 1 kafka Cluster

Generate cluster ID using script present in "/bin" folder:

```
1 bin/kafka-storage.sh random-uuid
```

Sample output: zJVVVZakS9aIBfjY0BIYJA

Map each node with this cluster id:

1. **Controller1:**

```
bin/kafka-storage.sh format -t zJVVVZAKS9aIBfjY0BIYJA -c config/controller1.properties
```

2. **Controller2:**

```
bin/kafka-storage.sh format -t zJVVVZAKS9aIBfjY0BIYJA -c config/controller2.properties
```

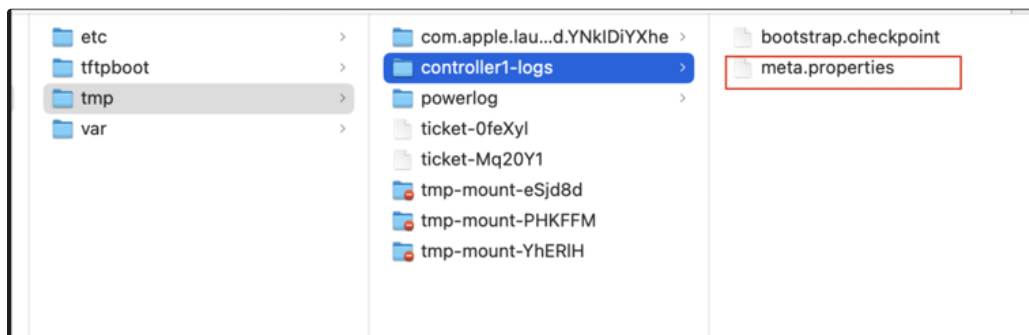
3. **Broker1:**

```
bin/kafka-storage.sh format -t zJVVVZAKS9aIBfjY0BIYJA -c config/broker1.properties
```

4. **Broker2:**

```
bin/kafka-storage.sh format -t zJVVVZAKS9aIBfjY0BIYJA -c config/broker2.properties
```

This will create "meta.properties" file within each log.dirs path.

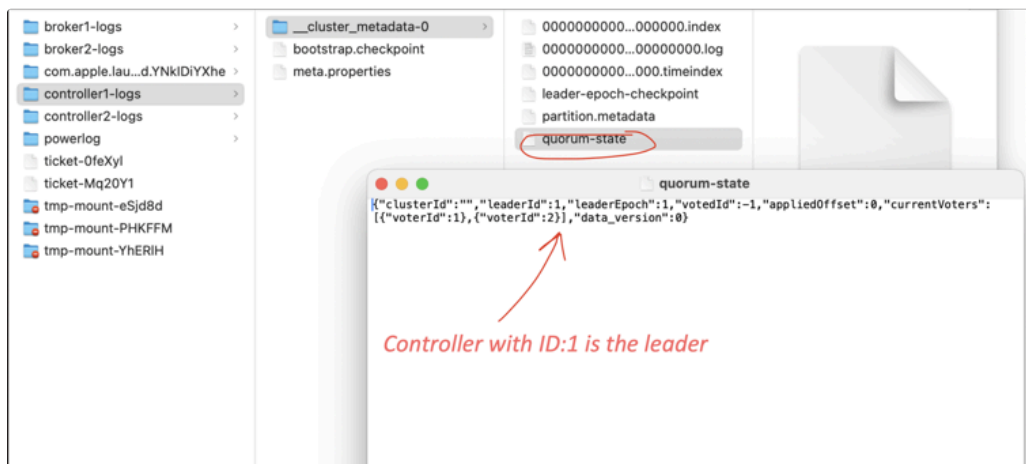


```
1 #
2
3 #Thu Feb 26 22:04:56 IST 2026
4
5 cluster.id=zJVVVZAKS9aIBfjY0BIYJA
6
7 directory.id=LYSuEb6ps8gnV7YsC9JHA
8
9 node.id=1
10
11 version=1
```

Step6: Start all the node servers

Order matters like : Controller first and then Broker. As broker try to call Controller to load the cluster metadata information.

```
1 Controller1:
2           bin/kafka-server-start.sh config/controller1.properties
3
4 Controller2:
5           bin/kafka-server-start.sh config/controller2.properties
6
```



Broker1:

```
1 bin/kafka-server-start.sh config/broker1.properties
```

```
(org.apache.kafka.common.config.AbstractConfig)
2026-02-26 22:13:08.139 INFO [BrokerServer id=3] Waiting for the broker to be unfenced (kafka.server.BrokerServer)
2026-02-26 22:13:08.180 INFO [BrokerLifecycleManager id=3] The broker has been unfenced. Transitioning from RECOVERY to RUNNING. (kafka.server.BrokerLifecycleManager)
2026-02-26 22:13:08.180 INFO [BrokerServer id=3] Finished waiting for the broker to be unfenced (kafka.server.BrokerServer)
2026-02-26 22:13:08.182 INFO [authorizerStart completed for endpoint BROKER. Endpoint is now READY. (org.apache.kafka.server.network.EndpointReadyFutures)
2026-02-26 22:13:08.182 INFO [SocketServer listenerType=BROKER, nodeId=3] Enabling request processing. (kafka.network.SocketServer)
2026-02-26 22:13:08.183 INFO [Awaiting socket connections on 0.0.0.0:9092. (kafka.network.DataPlaneAcceptor)
2026-02-26 22:13:08.184 INFO [BrokerServer id=3] Waiting for all of the authorizer futures to be completed (kafka.server.BrokerServer)
2026-02-26 22:13:08.184 INFO [BrokerServer id=3] Finished waiting for all of the authorizer futures to be completed (kafka.server.BrokerServer)
2026-02-26 22:13:08.184 INFO [BrokerServer id=3] Waiting for all of the SocketServer Acceptors to be started (kafka.server.BrokerServer)
2026-02-26 22:13:08.184 INFO [BrokerServer id=3] Finished waiting for all of the SocketServer Acceptors to be started (kafka.server.BrokerServer)
2026-02-26 22:13:08.185 INFO [BrokerServer id=3] Transition from STARTING to STARTED (kafka.server.BrokerServer)
2026-02-26 22:13:08.185 INFO Kafka version: 4.2.0 (org.apache.kafka.common.utils.AppInfoParser)
2026-02-26 22:13:08.185 INFO Kafka commitId: a18251baeb0825c6 (org.apache.kafka.common.utils.AppInfoParser)
2026-02-26 22:13:08.185 INFO Kafka startTimeMs: 1772124188185 (org.apache.kafka.common.utils.AppInfoParser)
2026-02-26 22:13:08.186 INFO [KafkaRaftServer nodeId=3] Kafka Server started (kafka.server.KafkaRaftServer)
```

Broker2:

```
1 bin/kafka-server-start.sh config/broker2.properties
```

```
(org.apache.kafka.common.config.AbstractConfig)
2026-02-26 22:13:10.836 INFO [BrokerServer id=4] Waiting for the broker to be unfenced (kafka.server.BrokerServer)
2026-02-26 22:13:10.876 INFO [BrokerLifecycleManager id=4] The broker has been unfenced. Transitioning from RECOVERY to RUNNING. (kafka.server.BrokerLifecycleManager)
2026-02-26 22:13:10.876 INFO [BrokerServer id=4] Finished waiting for the broker to be unfenced (kafka.server.BrokerServer)
2026-02-26 22:13:10.878 INFO [authorizerStart completed for endpoint BROKER. Endpoint is now READY. (org.apache.kafka.server.network.EndpointReadyFutures)
2026-02-26 22:13:10.879 INFO [SocketServer listenerType=BROKER, nodeId=4] Enabling request processing. (kafka.network.SocketServer)
2026-02-26 22:13:10.879 INFO [Awaiting socket connections on 0.0.0.0:9192. (kafka.network.DataPlaneAcceptor)
2026-02-26 22:13:10.881 INFO [BrokerServer id=4] Waiting for all of the authorizer futures to be completed (kafka.server.BrokerServer)
2026-02-26 22:13:10.881 INFO [BrokerServer id=4] Finished waiting for all of the authorizer futures to be completed (kafka.server.BrokerServer)
2026-02-26 22:13:10.881 INFO [BrokerServer id=4] Waiting for all of the SocketServer Acceptors to be started (kafka.server.BrokerServer)
2026-02-26 22:13:10.881 INFO [BrokerServer id=4] Finished waiting for all of the SocketServer Acceptors to be started (kafka.server.BrokerServer)
2026-02-26 22:13:10.881 INFO [BrokerServer id=4] Transition from STARTING to STARTED (kafka.server.BrokerServer)
2026-02-26 22:13:10.881 INFO Kafka version: 4.2.0 (org.apache.kafka.common.utils.AppInfoParser)
2026-02-26 22:13:10.881 INFO Kafka commitId: a18251baeb0825c6 (org.apache.kafka.common.utils.AppInfoParser)
2026-02-26 22:13:10.881 INFO Kafka startTimeMs: 1772124198881 (org.apache.kafka.common.utils.AppInfoParser)
2026-02-26 22:13:10.882 INFO [KafkaRaftServer nodeId=4] Kafka Server started (kafka.server.KafkaRaftServer)
```

Check the cluster status:

```
1 bin/kafka-metadata-quorum.sh --bootstrap-controller localhost:9093
describe --status
```

```
ClusterId: zJVWVZakS9aIBfjY0B1YJA
LeaderId: 1
LeaderEpoch: 1
HighWatermark: 719
MaxFollowerLag: 0
MaxFollowerLagTimeMs: 0
CurrentVoters: [{"id": 1, "endpoints": ["CONTROLLER://localhost:9093"]}, {"id": 2, "endpoints": ["CONTROLLER://localhost:9193"]}]]
CurrentObservers: [{"id": 4, "directoryId": "41f9OpGPB_YmhAerI4GCZA"}, {"id": 3, "directoryId": "Hczf6rWh7YvteyLMVvFVUg"}]]
shrayanshjain@Shrayanshs-MacBook-Pro kafka_2.13-4.2.0 %
```

controllers *brokers*

Step7: All setup completed, now lets test the Producer and consumer using script.

Lets create sample topic to test:

In this we are not passing, how many partitions and Replication factor, so Active controller use its default.

```
1 bin/kafka-topics.sh --bootstrap-server localhost:9092 --create --topic
  order-events-topic
```

--bootstrap-server localhost:9092, is only required for 1st time, because request need to go to some broker and that will ultimately pass to Leader Controller.

Once done, we can check the status of topic:

```
1 bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --
  topic order-events-topic
```

```
shrayanshjain@Shrayanshs-MacBook-Pro kafka_2.13-4.2.0 % bin/kafka-topics.sh --bootstrap-server localhost:9092 \
--describe --topic order-events-topic
Topic: order-events-topic TopicId: FGBARBA0QkC4MbwUuaNK2A PartitionCount: 3 ReplicationFactor: 2 Configs: min.insync.replicas=1,segment.bytes=1073741824
Topic: order-events-topic Partition: 0 Leader: 3 Replicas: 3,4 Isr: 3,4 Elr: LastKnownElr:
Topic: order-events-topic Partition: 1 Leader: 4 Replicas: 4,3 Isr: 4,3 Elr: LastKnownElr:
Topic: order-events-topic Partition: 2 Leader: 3 Replicas: 3,4 Isr: 3,4 Elr: LastKnownElr:
```

Step8: Producer test

```
1 bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic
  order-events-topic
```

```
shrayanshjain@Shrayanshs-MacBook-Pro kafka_2.13-4.2.0 % bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic order-events-topic
>hello
>this is shrayansh
>how are you
>
```

Step9: Consumer test

```
1 bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic  
order-events-topic --from-beginning
```

```
shrayanshjain@Shrayanshs-MacBook-Pro kafka_2.13-4.2.0 % bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic order-events-topic --from-beginning  
hello  
this is shrayansh  
how are you  
█
```